

• Sec-A ($2 \times 5 = 10$)

2. What is the use of turtle() method?

Ans The 'turtle()' method is not a standard method in Python. The 'turtle' module in Python's standard library, which provides a simple way to create graphics using a turtle metaphor. The turtle module allows us to create and control turtle graphics.

3. What is the primary purpose of the Turtle module in Python?

Ans The primary purpose of the Turtle module in Python is to provide a simple and intuitive way to create graphics and drawing using a turtle metaphor.

4. In the Turtle module, what does the forward() function do?

Ans In the Turtle module, the 'forward()' fun. moves the turtle object forward in the direction it is currently facing by a specified distance.

5. How can you bind a fun. to a Button widget in Tkinter?

Ans We can bind a function to a button widget in Tkinter using the 'bind()' method or by specifying the 'command' parameter when creating the button widget.

6. Which Tkinter geometry manager is based on placing widgets in a grid-like structure?

Ans The Tkinter geometry manager based on placing widgets in a grid-like structure is called the 'grid' geometry manager.

• Sec-B (7x3=21)

1. Explain the main components of a tkinter application and their roles in creating a graphical user interface (GUI).

Ans The main components of a Tkinter application and their roles in creating a graphical user interface (GUI) are:

• Tkinter Module :

The Tkinter module is the core components that provides the framework and tools for building GUI applications in Python. It is the standard GUI library for Python and comes pre-installed with most Python distributions.

- **Tk() :** The 'tk()' fun is used to initialize the Tkinter app. and create the main window or root window. This is the top-level container that holds all the other GUI elements.

- **Widgets :** widgets are the basic building blocks of a Tkinter GUI. They represent the various GUI elements like button, labels, text boxes, menus, etc. that the user can interact with. Some common Tkinter widgets include:-

- **Button :** Creates a clickable button.
- **Label :** Displays text or images.
- **Entry :** Allows user input through a text field.
- **Frame :** Acts as a container to group and organize other widgets.

- **Geometry Managers :** Geometry managers control the placement and layout of widgets within the GUI. Tkinter provides three main geometry managers:-

- **Pack() :-** Organize widgets in blocks, filling available space.
- **grid() :-** Arranges widgets in a table-like grid structure.

- `Place()` :- Allows explicit positioning of widgets by specifying coordinates.

- **Event Handling :-**

Tkinter allows ~~use~~ to bind functions to widget events, such as button clicks, key presses, mouse movement, etc. This enables to add interactivity and functionality to our GUI application.

- **Mainloop()** :- The 'mainloop()' fun. is used to start the Tkinter event loop, which keeps the application running and responsive to user interactions until the window is closed.

2. Describe the basic workflow of drawing with the Turtle module in Python. include the steps involved from initializing the Turtle to drawing shapes on the screen.

Ans The basic workflow of drawing with the Turtle module in Python involves several steps :-

(i) **Import The Turtle module :-** Start by importing the 'Turtle' module at the beginning of your Python script.

```
import turtle
```

(ii) **Create a turtle screen :-** Create a turtle screen object, which serves as the drawing canvas. ~~then~~ we can customize the screen's size, background color and other properties if needed.

```
screen = turtle.Screen()
```

- (iii) Create a turtle object:- Create a turtle object, which represent the drawing turtle. We can customize the turtle's appearance, such as its shape, color, and speed.

```
t = turtle.Turtle()
```

- (iv) Draw shapes and lines: Use the turtle object ('t') to draw shapes, lines, and patterns on the screen. We can control the turtle's movements and actions using various methods such as 'forward()', 'backward()', 'left()', 'right()', 'circle()', 'dot()', etc.

```
t.forward(100)
```

```
t.left(90)
```

```
t.forward(100)
```

- (v) Customize drawing attributes: ~~you~~^{we} can customize the drawing attributes such as the pen color, pen size, fill color, etc., before drawing shapes using the turtle objects methods like 'color()', 'pensize()', 'fillcolor()', etc.

```
t.color("red")
```

```
t.pensize(2)
```

```
t.fillcolor("blue")
```

- (vi) Control the turtle:- We can control the turtle's behaviour using various methods such as, 'speed()', 'penup()', 'pendown()', etc., to change its speed, lift or lower the pen, etc.

```
t.speed(1)
```

```
t.penup()
```

```
t.goto(0,0)
```

```
t.pendown()
```

- vii) close the turtle graphics window : once we are done drawing, close the turtle graphics window to display the final drawing.

Screen.mainloop()

By above following these steps, we can create drawings and graphic using the turtle module in Python.

3. Describe the process of creating a new window in Tkinter. Include the necessary steps and code examples.

Ans Creating a new window in Tkinter involves a few essential steps. Below is a breakdown of the process along with code examples:

- **Step 1: import the Tkinter module:**
First we need to import the Tkinter module in our python script. This module provides the necessary functions and classes for creating GUI applications.

#code:

```
import tkinter as tk
```

- **Step 2: Create the main Tkinter window:-**

Next, we are create the main tkinter window. This window will serve as the parent window for any additional window which we ~~are~~ create.

#code:

```
root = tk.TK()
```

```
root.title("Main Window")
```

- Step 3: Define a function to create a new window
We define a function that will create a new window when called. This function will typically contain the code to create and configure the new window.

ex

```
def create_new_window():
    new_window = tk.Toplevel(root)
    new_window.title("New Window")
```

- Step 4: Add a button to trigger the new window creation.
We add a button to the main window that, when clicked, will call the function to create a new window.

ex:

```
button = tk.Button(root, text="Open New Window",
                    command=create_new_window)
button.pack()
```

- Step 5: Run the Tkinter main loop
Finally, we need to start the Tkinter main loop to display the windows and handle user interactions.

ex

```
root.mainloop()
```

By following these steps and incorporating the provided code examples, we can successfully create a new window in Tkinter.

- Sec - C ($1 \times 3 = 33$)

1. Discuss the differences between the Pack, Grid, and place geometry managers in Tkinter. Provide scenario where each manager would be most suitable.

Ans

Tkinter offers three main geometry managers for arranging widgets within our GUI: pack, grid, and place. Each has its strengths and weaknesses:

Pack:

Concept: pack organizes widgets in horizontal or vertical boxes along ~~with~~ the sides (top, down, left, right) of a container. It's like stacking items in a box.

- Advantages:

- Easiest to use for simple layouts.
- Adapts well to resizing windows.

- Disadvantages:

- Limited control over precise placement.
- Can lead to unexpected behaviour with complex layouts.

- Suitable for:

- Simple toolbars with buttons or labels arranged horizontally or vertically.
- Basic layouts with a single widget filling the container or two widgets stacked on top of each other.

Grid:

Concept: ~~Grid positions~~ The Grid geometry manager arranges widgets in rows and columns, allowing more complex and precise layouts.

- Suitability:

- ~~Best suited for simple~~
- Ideal for organizing widgets in a tabular format with rows and columns.

- Suitable for layout requiring widgets to span multiple rows or columns.

Place: The Place geometry manager allows absolute positioning of widgets by specifying their exact coordinates.

- Suitability:
 - Useful for creating custom or highly specialized layouts where precise control over widget positioning is required.
 - Suitable for creating graphical interfaces with elements overlaying each other.

2. Discuss the importance of loops in graphics programming with the Turtle module. Provide examples of how loops can be used to simplify repetitive tasks such as drawing multiple shapes or patterns.

Ans Loops are essential in graphics programming with the Turtle module because they allow to simplify repetitive tasks by automating the execution of commands. The turtle module provides a simple and intuitive way to create graphics by controlling a turtle that moves on the screen, drawing lines as it moves. Loops can be used to repeat drawing commands for shapes or patterns, reducing code duplication and making the code more efficient and maintainable.

- Examples of loops in Turtle Graphics -

(i) Drawing Shapes: Imagine drawing a square. We would use "forward(distance)" four times to create the sides. With a loop, we can achieve this efficiently:-

```

ex
for _ in range(4):
    forward(distance)
    left(90)

```

- (ii) Drawing Patterns: Loops can also be used to create patterns by repeating a sequence of movements that define the pattern. For instance, to draw a spiral pattern, we can use a loop to move the turtle forward and turn it right by a small angle repeatedly:

```

ex
for _ in range(100):
    turtle.forward(1)
    turtle.right(0.1)

```

- (iii) Drawing Polygons: Loops are used to draw polygons. For ex. to draw a hexagon, we can use a loop to move the turtle forward and turn it right by 60 degrees six times:

```

ex
for _ in range(6):
    turtle.forward(100)
    turtle.right(60)

```

Overall, loops are essential in graphics programming with the Turtle module as they enable the repetition of actions, simplify repetitive tasks, and make the code more concise and efficient.

- Q3. Write a tkinter program that creates a simple Calculator GUI with buttons for digits (0-9), arithmetic operators (+, -, *, /) and an entry widget to display the result. Include the necessary event handling for button clicks and result calculation.

May 8, 2024, 00:29

DATE: / /

PAGE NO.: 57

```
Ans import tkinter as tk

def button_click(number):
    current = entry.get()

    root = tk.Tk()
    root.geometry("250x250")
    root.title("Simple Calculator")

    display = tk.StringVar()
    display.set("0")

    display_label = tk.Label(root, textvariable=display, font=("Arial", 24),
                             anchor="e", background="white")
    display_label.grid(row=0, column=0, columnspan=4)

    buttons = [
        "7", "8", "9", "/",
        "4", "5", "6", "*",
        "1", "2", "3", "-",
        "0", ".", "=", "+"
    ]

    row_val = 1
    col_val = 0

    for button in buttons:
        tk.Button(root, text=button, width=5, command=lambda
            button=button: click_button(button)).grid(row_val, column=col_val)
        col_val += 1
    if col_val > 3:
        col_val = 0
        row_val += 1
```

```

def click-button(button):
    if button == "=":
        try:
            result = eval(display.get())
            display.set(str(result))
        except Exception as e:
            display.set("Error")
    else:
        if display.get() == "0":
            display.set(button)
        else:
            display.set(display.get() + button)

```

root.mainloop()

Output:

32*6			
7	8	9	/
4	5	6	*
1	2	3	-
0	.	=	+

Simple Calculator (GUI)

22BCON1068